

UK PATENT APPLICATION

SYSTEM AND METHOD FOR AUTONOMOUS CROSS-DOMAIN CAUSAL DISCOVERY AND BUSINESS GENERATION USING ENSEMBLE PREDICTION AND TEST-DRIVEN DEVELOPMENT

DESCRIPTION

TECHNICAL FIELD

The present invention relates to computer-implemented systems for autonomous cross-domain causal discovery, statistical prediction, and automated business generation. More particularly, the invention relates to a closed-loop platform that ingests heterogeneous time-series data from multiple external sources, discovers causal relationships between behavioural signals and macroeconomic outcomes at individual variable granularity, generates ensemble predictions combining multiple statistical models with automatic weight calibration, autonomously converts statistically significant causal signals into deployed web applications through test-driven development, measures real-world outcomes against predictions, and recalibrates the system accordingly - all without human intervention.

BACKGROUND ART

The discovery of actionable relationships between disparate data domains and the exploitation of those relationships through commercial products presently requires coordination across multiple disconnected technological disciplines. The following categories of existing systems each address isolated segments of the problem but fail to provide an integrated autonomous solution.

Quantitative Trading Platforms such as Alpaca, QuantConnect, and Interactive Brokers provide algorithmic trading capability based on statistical signals. These systems predict price movements and execute trades but do not discover cross-domain causal relationships outside financial markets, do not generate software products to exploit discovered opportunities, and do not operate autonomously across the full cycle from data ingestion to product deployment. The fundamental limitation is confinement to a single domain (financial markets) and a single action type (trade execution).

AI Code Generation Systems such as GitHub Copilot, Devin (Cognition Labs), and Replit Agent generate software code from natural language descriptions. These systems build software artefacts but do not independently discover what should be built, do not identify statistical signals warranting product creation, and rely entirely on human specification of requirements. The fundamental limitation is the absence of autonomous opportunity discovery - a human must decide what to build and why.

Business Intelligence Platforms such as Palantir Foundry, Tableau, and Power BI correlate variables across datasets and present analytical dashboards. These systems identify correlations but do not test for temporal causal direction, do not generate predictions, do not score or rank opportunities for exploitation, and do not create products or take autonomous action. The fundamental limitation is confinement to retrospective analysis without forward-looking prediction or autonomous execution.

Automated Machine Learning Platforms such as H2O.ai, DataRobot, and Google AutoML automate model selection and training for prediction tasks. These systems generate and evaluate statistical models but require human-defined target variables, do not discover which variables are worth predicting, do not generate deployable products from model outputs, and do not close the loop between prediction and outcome measurement. The fundamental limitation is dependence on human problem formulation and absence of end-to-end autonomy.

In the patent literature, various publications describe components of the above systems in isolation. Patent publications relating to automated machine learning describe systems for model selection and hyperparameter tuning (see, for example, published applications by DataRobot, Inc. and H2O.ai, Inc.). Patent publications in the field of algorithmic trading describe signal-based trade execution systems confined to financial markets. Patent publications relating to AI-assisted software development describe code completion and generation tools that respond to human-authored specifications. Patent publications in the field of causal inference describe methods for identifying causal relationships within single datasets. However, no prior art patent publication or combination of publications discloses or suggests a system that integrates cross-domain causal discovery at individual variable granularity, multi-model ensemble prediction with empirical weight calibration, autonomous signal-to-software conversion via test-driven development, and closed-loop outcome measurement with model recalibration in a single autonomous pipeline.

There exists a need for a computer-implemented system that integrates the following capabilities absent from all prior art: (1) variable-level cross-domain causal discovery across heterogeneous data sources, (2) multi-model ensemble prediction with empirical weight calibration, (3) autonomous conversion of statistical signals into deployed software products, (4) closed-loop outcome measurement and system recalibration, and (5) architectural enforcement preventing trivial same-domain correlations from consuming system resources.

SUMMARY OF INVENTION

The present invention provides a computer-implemented system for autonomous cross-domain causal discovery and business generation that addresses the aforementioned limitations of prior art systems. The system comprises the following novel technical contributions:

Innovation (1): Variable-Level Cross-Domain Causal Discovery. Unlike existing systems that correlate aggregate domain-level metrics, the present invention operates at individual variable granularity, computing pairwise statistical associations across all ingested variables from at least six heterogeneous external sources, yielding thousands of

unique variable pairs. Bidirectional Granger causality testing with frequency-adaptive lag selection determines temporal causal direction at the individual variable level rather than at the domain level.

Innovation (2): Three-Model Ensemble with Automatic Weight Calibration. The system combines Granger causality predictions (default weight zero point four zero), ordinary least squares regression (default weight zero point three five), and autoregressive integrated moving average forecasting (default weight zero point two five) into a single ensemble prediction. Unlike fixed-weight ensemble systems, the present invention calibrates weights from historical prediction accuracy requiring a minimum of ten validated predictions, normalising per-model accuracy scores to derive updated weights.

Innovation (3): Autonomous TDD-Driven MVP Generation from Statistical Signals. The system converts statistical causal signals directly into deployed web applications via a five-phase test-driven development pipeline: specification generation, failing test creation (RED), implementation generation (GREEN), code refinement (REFACTOR), and output validation (VALIDATE). The pipeline accepts implementations achieving at least eighty percent test pass rate and supports up to five retry attempts with failure context carried forward.

Innovation (4): Closed-Loop Autonomous Business Pipeline. The system operates a complete autonomous cycle: data ingestion, correlation analysis, Granger causality testing, ensemble prediction, exploitation scoring, MVP code generation, cloud deployment, outcome measurement, and weight recalibration - executing on a daily scheduled cycle without human intervention. This represents the first integration of statistical discovery, product generation, and empirical validation in a single autonomous system.

Innovation (5): Cross-Layer Enforcement Architecture. The system enforces that behavioural signal variables (Layer 1: Wikipedia pageviews, Reddit activity) predict exploitable macroeconomic target variables (Layer 2: Federal Reserve economic indicators, stock prices, academic output, geopolitical events). This architectural constraint is enforced at the database query level, preventing same-layer correlations from entering the prediction pipeline and ensuring all predictions represent genuine cross-domain causal hypotheses.

Innovation (6): Build-Iterate Loop with Failure Diagnosis. When a generated application fails any test-driven development phase, the system diagnoses the failure mode, categorises errors across seven error categories (syntax, test, frontend, import, wiring, configuration, runtime), generates an iteration reason, and spawns a child build with adjusted parameters - closing the loop between build failure and corrective action without human intervention.

BRIEF DESCRIPTION OF THE DRAWINGS

Figure 1: System Architecture Overview diagram showing the complete autonomous pipeline from data ingestion through deployment and outcome measurement with nine processing stages connected by data flow arrows per Claims 1, 11, 16, with numbered annotation indicating Innovation (4) (closed-loop autonomous business pipeline).

Figure 2: Multi-Source Data Ingestion Pipeline diagram showing six external data sources (Wikipedia, Reddit, FRED, Stock, ArXiv, GDELТ) feeding into a standardisation layer that produces unified TimeSeriesData records per Claims 1, 2, 11, with numbered annotation indicating Innovation (1) (variable-level cross-domain data collection).

Figure 3: Cross-Domain Correlation Engine diagram showing NxN pairwise correlation computation with Pearson, Spearman, and Kendall coefficients, significance filtering at $p < 0.05$, and cross-domain enforcement excluding same-source pairs per Claims 1, 3, 11, with numbered annotations indicating Innovation (1) (variable-level discovery) and Innovation (5) (cross-layer enforcement).

Figure 4: Bidirectional Granger Causality System diagram showing forward test ($X \rightarrow Y$) and reverse test ($Y \rightarrow X$) with frequency-adaptive lag selection, direction assignment logic (`x_to_y`, `y_to_x`, `bidirectional`, `none`), and minimum thirty-point sample requirement per Claims 1, 4, 11, with numbered annotation indicating Innovation (1) (causal direction determination).

Figure 5: Three-Model Ensemble Architecture diagram showing Granger predictor (weight 0.40), OLS predictor (weight 0.35), and ARIMA predictor (weight 0.25) feeding into weighted voting combination with `up_score` and `down_score` computation, feature engineering block (rolling windows 7, 30, 90), and weight calibration feedback arrow per Claims 1, 5, 6, 11, 12, with numbered annotation indicating Innovation (2) (ensemble with automatic calibration).

Figure 6: Cross-Layer Enforcement diagram showing Layer 1 sources (Wikipedia, Reddit) constrained to predict Layer 2 targets (FRED, Stock, ArXiv, GDELТ) with SQL-level JOIN filter preventing same-layer pairs, directional arrow indicating signal->target assignment per Claims 1, 7, 11, with numbered annotation indicating Innovation (5) (architectural enforcement).

Figure 7: Exploitation Scoring Pipeline diagram showing five weighted components (Demand 30, Growth 25, Timing 20, Evidence 15, Competition 10) feeding into `opportunity_score` calculation, with action type classification (BUY, SELL, BUILD, MONITOR) and build viability threshold per Claims 1, 8, 11, with numbered annotations indicating Innovation (2) (scoring) and Innovation (4) (autonomous action).

Figure 8: Autonomous TDD Code Generation Overview diagram showing specification input flowing through five phases (SPEC->RED->GREEN->REFACTOR->VALIDATE) with AI language model interaction at RED, GREEN, and REFACTOR stages, eighty percent pass rate threshold at GREEN, and LIVE/FAILED output per Claims 1, 9, 11, 13, 16, 28, with numbered annotation indicating Innovation (3) (signal-to-software).

Figure 9: Five-Phase TDD Pipeline Detail diagram showing SPEC phase (YAML generation with complexity tiers LOW/MEDIUM/HIGH), RED phase (test creation with stub module), GREEN phase (AI implementation with five-retry loop and pass rate computation), REFACTOR phase (quality improvement with regression check), and VALIDATE phase (AST parsing, compilation, final test run) per Claims 9, 10, 13, 14, 28, with numbered annotation indicating Innovation (3) (TDD pipeline specifics).

Figure 10: Deployment Automation Flow diagram showing GitHub repository creation (PyGithub, Git Tree API, commit SHA verification) and Railway cloud deployment

(GraphQL mutations: projectCreate->serviceCreate->serviceInstanceDeploy) with status polling and stale-build detection at six hundred seconds per Claims 1, 9, 11, 15, 16, with numbered annotations indicating Innovation (3) (autonomous deployment) and Innovation (4) (no human intervention).

Figure 11: Outcome Measurement and Feedback Loop diagram showing prediction validation (pending->validated status transition, direction correctness, percentage error), walk-forward backtesting (expanding window OLS), exploitation backtesting (opportunity window measurement), and weight recalibration arrow feeding back to ensemble engine per Claims 1, 5, 6, 11, 12, 16, with numbered annotations indicating Innovation (2) (weight calibration) and Innovation (4) (closed-loop validation).

Figure 12: Build-Iterate Loop diagram showing failure diagnosis (seven error categories: syntax, test, frontend, import, wiring, config, runtime), iteration context generation (error patterns + parent files + ensemble data), child build spawning (parent_build_id linkage, iteration_number increment), and concurrency guard (block if active build in chain) per Claims 28, 29, 30, 31, with numbered annotation indicating Innovation (6) (self-healing pipeline).

Figure 13: Autonomous Scheduling Timeline diagram showing daily (01:00 Wikipedia, 01:30 Reddit, 02:00 GDELT, 02:30 update actuals, 05:00 ensemble predictions) and weekly (Sunday 03:00 baselines, Monday 04:00 walk-forward) cron schedule with data dependency arrows showing ingestion->validation->prediction ordering per Claims 1, 11, 16, with numbered annotation indicating Innovation (4) (autonomous scheduling).

Figure 14: End-to-End Example Scenario diagram showing Wikipedia "Artificial_intelligence" pageviews (Layer 1) -> correlation $r=0.72$ -> Granger $p=0.03$ at lag 3 -> ensemble prediction "up" confidence 0.765 -> exploitation score 72 -> TDD build -> GitHub push -> Railway deploy -> three-month outcome validation, illustrating all six innovations ((1)(2)(3)(4)(5)(6)) operating in sequence per Claims 1, 11, 16.

DETAILED DESCRIPTION

The following detailed description refers to the accompanying drawings (Figures 1 through 14) and describes preferred embodiments of the invention. Those skilled in the art will recognise that the invention may be embodied in many different forms and should not be construed as limited to the embodiments described herein.

System Architecture Overview

Referring to Figure 1, the system comprises a server executing a FastAPI web application framework connected to a PostgreSQL relational database via SQLAlchemy object-relational mapping. The server hosts a scheduler module (APScheduler) that triggers autonomous pipeline stages on configurable cron schedules. The system further comprises a data ingestion module, a correlation analysis engine, a Granger causality testing module, an ensemble prediction engine, an exploitation scoring module, a specification generation module, a test-driven development orchestrator, a GitHub integration service, a Railway deployment service, an outcome measurement module, and a walk-forward backtesting module.

The server hardware requirements comprise a processor capable of executing concurrent background tasks, sufficient memory to hold aligned time-series data for pairwise statistical computation (typically requiring aligned arrays of at least thirty data points per pair for Granger testing and at least twenty data points per pair for correlation), non-volatile storage for the PostgreSQL database containing time-series records, variable metadata, correlation results, prediction tracking, exploitation recommendations, and build records, and a network interface for communicating with external data source APIs, cloud deployment platforms, and code hosting services.

The frontend comprises server-rendered HTML templates (Jinja2) with Alpine.js for reactive client-side state management. The dashboard provides real-time visibility into all pipeline stages: data freshness, correlation matrices, Granger causality results, ensemble predictions, exploitation recommendations, and build portfolio status. The dashboard interface presents sortable and paginated tabular displays for each data domain, enabling human operators to monitor autonomous system performance without intervening in pipeline execution.

The system communicates with external services via RESTful APIs (data sources, deployment platforms), GraphQL API (Railway cloud platform), and the PyGithub library (GitHub repository management). All pipeline operations execute server-side as background tasks using FastAPI's BackgroundTasks mechanism, ensuring that client disconnection does not interrupt long-running autonomous processes. Each pipeline stage is idempotent - repeated execution produces equivalent results - enabling fault recovery through simple re-execution of the scheduled task.

The system's nine-stage pipeline, illustrated in Figure 1, operates as follows: (1) multi-source data ingestion collects raw time-series data from external APIs; (2) standardisation normalises all records into a common format comprising variable identifier, source tag, timestamp, and numerical value; (3) the correlation engine computes pairwise statistical associations across all active variables; (4) the Granger causality module tests temporal causal direction for significant cross-domain pairs; (5) the ensemble prediction engine generates weighted multi-model predictions for qualifying pairs; (6) the exploitation scoring module ranks discovered opportunities and assigns action types; (7) the TDD code generation pipeline converts BUILD-type recommendations into tested software artefacts; (8) the deployment module pushes generated code to GitHub and deploys it to Railway cloud infrastructure; and (9) the outcome measurement module validates predictions against realised values and recalibrates model weights. The feedback arrow from stage 9 back to stage 5 represents the closed-loop recalibration mechanism that distinguishes this system from prior art approaches.

Multi-Source Data Ingestion Pipeline

Referring to Figure 2, the data ingestion pipeline (Innovation (1)) collects time-series data from at least six heterogeneous external sources spanning behavioural and macroeconomic domains. Each data source is mapped to a standardised internal representation comprising a variable identifier, a data source tag, a timestamp, and a numerical value.

The system ingests data from the following source categories:

Layer 1 (Behavioural Signals):

- **Wikipedia pageview data:** Daily and monthly pageview counts for specified article topics, providing behavioural demand signals. Ingestion is scheduled at 01:00 UTC daily.
- **Reddit activity data:** Post counts, comment volumes, and engagement metrics for specified subreddits over a thirty-day rolling window. Ingestion is scheduled at 01:30 UTC daily.

Layer 2 (Exploitable Outcomes):

- **Federal Reserve Economic Data (FRED):** Macroeconomic indicators including interest rates, inflation metrics, employment figures, and GDP components, providing economic outcome variables.
- **Stock market data:** Price and volume data for specified equities and exchange-traded funds, providing financial outcome variables.
- **GDELT event data:** Geopolitical event counts and average tone metrics over a thirty-day window, providing geopolitical outcome variables. Ingestion is scheduled at 02:00 UTC daily.
- **ArXiv publication data:** Publication counts and citation metrics for specified academic categories, providing academic output variables.

All ingested data is stored in a TimeSeriesData table with foreign key reference to a VariableMetadata table that records the variable name, data source identifier, description, units, and ingestion frequency. The VariableMetadata table currently contains sixty-one active variables, yielding three thousand seven hundred and twenty-one unique pairwise combinations.

Cross-Domain Correlation Engine

Referring to Figure 3, the correlation analysis engine (Innovation (1), Innovation (5)) computes the complete NxN upper-triangle pairwise correlation matrix across all active variables. For each pair (i, j) where $i < j$, the engine performs the following steps:

1. **Data retrieval:** Fetch time-series records for both variables from the TimeSeriesData table.
2. **Temporal alignment:** Perform outer-join alignment on timestamps with linear interpolation to fill gaps, ensuring equal-length series for statistical computation.
3. **Sample size verification:** Verify that the aligned series contains at least twenty data points. Pairs failing this threshold are skipped.
4. **Correlation computation:** Calculate three correlation coefficients:
 - Pearson product-moment correlation coefficient (linear relationship strength)
 - Spearman rank-order correlation coefficient (monotonic relationship strength)
 - Kendall tau-b correlation coefficient (concordance-based relationship strength)
5. **Significance testing:** Compute two-tailed p-values for each coefficient. A pair is marked as significant if $p < 0.05$.

6. Cross-domain enforcement (Innovation (5)): Verify that the two variables originate from different data source categories. Same-source pairs are excluded from downstream processing.

7. Storage: Persist correlation value, absolute correlation (for ranking), p-value, sample size, and source identifiers in the CorrelationResult table.

The engine processes the full matrix on each scheduled run, enabling detection of emerging correlations as new data accumulates.

The cross-domain correlation computation represents a significant departure from prior art approaches. Existing business intelligence platforms such as Palantir and Tableau compute correlations within a single dataset or between pre-selected variables. The present invention computes the complete NxN matrix across all active variables from all sources simultaneously, enabling the discovery of unexpected cross-domain relationships that no human analyst has hypothesised. The three-coefficient approach (Pearson, Spearman, Kendall) captures both linear and non-linear monotonic relationships, and the minimum twenty-point requirement ensures statistical validity while the cross-domain filter (Innovation (5)) prevents the system from wasting computational resources on trivially correlated same-source variables.

Algorithm: Pairwise Correlation Computation

Input: active_variables (list of VariableMetadata records)

Output: correlation_results (list of CorrelationResult records)

Step 1: For each pair (i, j) where $i \leq j$ in active_variables:

- a. Retrieve time_series_i from TimeSeriesData where variable_id = i
- b. Retrieve time_series_j from TimeSeriesData where variable_id = j
- c. Perform outer-join alignment on timestamps
- d. Apply linear interpolation to fill gaps in aligned series
- e. If $\text{len}(\text{aligned_series}) < 20$: skip pair, continue
- f. Compute pearson_r , $\text{pearson_p} = \text{pearsonr}(\text{values_i}, \text{values_j})$
- g. Compute spearman_r , $\text{spearman_p} = \text{spearmanr}(\text{values_i}, \text{values_j})$
- h. Compute kendall_tau , $\text{kendall_p} = \text{kendalltau}(\text{values_i}, \text{values_j})$
- i. Set $\text{is_significant} = \text{True}$ if $\text{minimum}(\text{pearson_p}, \text{spearman_p}, \text{kendall_p}) < 0.05$
- j. Set $\text{abs_correlation} = \text{abs}(\text{pearson_r})$ for ranking
- k. Retrieve $\text{source_i} = \text{variable_i.source}$, $\text{source_j} = \text{variable_j.source}$
- l. If $\text{source_i} == \text{source_j}$: mark as same-source, exclude from downstream
- m. Persist CorrelationResult with all computed values

Step 2: If pair has $\text{sample_size} \geq 30$ AND is_significant :

- a. Proceed to Granger causality testing (Figure 4)

Bidirectional Granger Causality System

Referring to Figure 4, the Granger causality module (Innovation (1)) tests temporal causal direction for all significant cross-domain correlation pairs. For each qualifying pair, the module performs bidirectional testing:

Forward test (X -> Y): Tests whether past values of variable X improve prediction of variable Y beyond what past values of Y alone provide. The null hypothesis H_0 states: "X does not Granger-cause Y."

Reverse test (Y → X): Tests whether past values of variable Y improve prediction of variable X beyond what past values of X alone provide. The null hypothesis H0 states: "Y does not Granger-cause X."

Frequency-Adaptive Lag Selection: The maximum number of lags tested is determined by the data frequency of the pair:

- Daily frequency: maximum two hundred and fifty-two lags (one year of trading days)
- Weekly frequency: maximum fifty-two lags (one year)
- Monthly frequency: maximum twelve lags (one year)
- Quarterly frequency: maximum four lags (one year)
- Yearly frequency: maximum two lags (two years)

A minimum buffer of five to ten periods (frequency-dependent) is enforced to ensure sufficient degrees of freedom. The system requires a minimum of thirty aligned data points for Granger testing.

Causal Direction Assignment: Based on the bidirectional test results at significance threshold $p < 0.05$, the system assigns one of four causal direction categories:

- **x_to_y:** Forward test significant, reverse test not significant. Variable X Granger-causes variable Y.
- **y_to_x:** Reverse test significant, forward test not significant. Variable Y Granger-causes variable X.
- **bidirectional:** Both tests significant. Mutual Granger causality detected.
- **none:** Neither test significant. No Granger causal relationship detected.

The Granger p-values (forward and reverse), optimal lag count, and causal direction are stored in the CorrelationResult table alongside the correlation coefficients, creating a unified statistical profile for each variable pair.

Three-Model Ensemble Prediction Engine

Referring to Figure 5, the ensemble prediction engine (Innovation (2)) combines three complementary statistical models into a weighted prediction for each qualifying signal-target pair.

Sub-Model 1: Granger Causality Predictor (Default Weight: 0.40)

The Granger predictor leverages the pre-computed causal direction and optimal lag from the Granger causality module. Given a signal-target pair where the signal Granger-causes the target at optimal lag k , the predictor:

1. Retrieves the most recent k values of the signal variable.
2. Computes the signal trend over the lag window.
3. Predicts direction: if the signal has increased over the lag window, the target is predicted to increase (and vice versa).

4. Confidence is derived from the Granger p-value: $\text{confidence} = 1 - p_value$, bounded to $[0, 1]$.

Sub-Model 2: Ordinary Least Squares Regression (Default Weight: 0.35)

The OLS predictor uses an expanding-window walk-forward approach:

1. Requires a minimum of six months of aligned training data ($\text{MIN_OLS_TRAIN_MONTHS} = 6$).
2. Fits a linear regression model: $\text{target} = \text{slope} \times \text{signal} + \text{intercept}$.
3. Predicts direction from the sign of the slope coefficient.
4. Confidence is derived from the R^2 coefficient of determination, bounded to $[0, 1]$.

Sub-Model 3: Autoregressive Integrated Moving Average (Default Weight: 0.25)

The ARIMA predictor uses a univariate ARIMA(1,1,1) model on the target variable:

1. Requires a minimum of twenty-four data points ($\text{MIN_ARIMA_POINTS} = 24$).
2. Fits ARIMA with order (1, 1, 1) - one autoregressive term, one differencing, one moving average term.
3. Produces a one-step-ahead forecast.
4. Direction is determined by comparing the forecast to the last observed value.
5. Confidence is derived from the forecast standard error, normalised and bounded to $[0, 1]$.

Ensemble Combination Algorithm:

Given three sub-model outputs, each producing a direction (up or down) and a confidence score, the ensemble combines them as follows:

```
up_score = Sum(weight_i x confidence_i) for all models predicting "up"
down_score = Sum(weight_i x confidence_i) for all models predicting "down"
final_direction = "up" if up_score >= down_score, otherwise "down"
final_confidence = min(|up_score - down_score| / total_weight, 1.0)
```

Where $\text{total_weight} = \text{Sum}(\text{weight_i})$ for all contributing models (models that successfully produced a prediction).

Feature Engineering: Prior to prediction, the system engineers features from the signal time-series using three rolling window sizes: seven, thirty, and ninety periods. For each window, the system computes the rolling mean, rolling standard deviation, and rolling coefficient of variation (volatility).

Automatic Weight Calibration (Innovation (2)):

When at least ten validated predictions exist for a given signal-target pair, the system recalibrates sub-model weights based on historical per-model accuracy:

1. Query all PredictionTracking records where status = 'validated' for the pair.
2. For each sub-model, compute $\text{accuracy} = \text{correct_predictions} / \text{total_predictions}$.

3. Normalise: $\text{calibrated_weight}_i = \text{accuracy}_i / \text{Sum}(\text{accuracy}_j)$ for all models.
4. Apply calibrated weights for subsequent predictions of this pair.

This mechanism enables the system to automatically favour whichever sub-model has demonstrated the highest empirical accuracy for each specific signal-target pair, rather than relying on fixed default weights.

Algorithm: Ensemble Prediction Generation

Input: signal_name (string), target_name (string), weights (dict)

Output: prediction (dict with direction, confidence, change_pct, sub_models)

Step 1: Variable Resolution

- a. Retrieve signal_var from VariableMetadata where name = signal_name
- b. Retrieve target_var from VariableMetadata where name = target_name
- c. If either not found: return error

Step 2: Data Loading

- a. Load signal_ts from TimeSeriesData where variable_id = signal_var.id
- b. Load target_ts from TimeSeriesData where variable_id = target_var.id
- c. If len(target_ts) < MIN_OLS_TRAIN_MONTHS (6): return error

Step 3: Feature Engineering

- a. For each window_size in [7, 30, 90]:
 - i. Compute rolling_mean = mean(signal_ts[-window_size:])
 - ii. Compute rolling_std = std(signal_ts[-window_size:])
 - iii. Compute rolling_cv = rolling_std / rolling_mean if rolling_mean != 0

Step 4: Sub-Model Execution

- a. granger_result = _granger_predict(signal_var, target_var, signal_ts, target_ts)
Returns: {direction, confidence, lag_used, p_value}
- b. ols_result = _ols_predict(signal_ts, target_ts)
Returns: {direction, confidence, slope, intercept, r_squared}
- c. arima_result = _arima_predict(target_ts)
Returns: {direction, confidence, forecast_value, std_error}

Step 5: Weight Calibration (if sufficient history)

- a. Query PredictionTracking where signal=signal_name AND target=target_name AND status='validated'
- b. If count >= 10:
 - i. For each model_version in ['granger_v1', 'ols_v1', 'arima_v1']:

accuracy = count(direction_correct=True) / count(total)
 - ii. total_accuracy = sum(all accuracies)
 - iii. calibrated_weight_i = accuracy_i / total_accuracy
 - iv. Replace default weights with calibrated weights

Step 6: Ensemble Combination

- a. Initialize up_score = 0, down_score = 0
- b. For each sub_model in [granger, ols, arima]:
 - i. If sub_model produced valid result:

If direction == "up": up_score += weight_i x confidence_i

Else: down_score += weight_i x confidence_i
- c. total_weight = sum(weight_i for all contributing models)
- d. final_direction = "up" if up_score >= down_score else "down"
- e. final_confidence = min(abs(up_score - down_score) / total_weight, 1.0)
- f. predicted_change_pct derived from weighted average of sub-model magnitudes

Step 7: Persistence

- a. Create PredictionTracking record with:

prediction_id = UUID, signal_name, target_name,

predicted_direction, predicted_change_pct, confidence,

model_version = 'ensemble_v1', status = 'pending',

target_date = current_date + optimal_lag

b. Return prediction dict with all sub-model details

The ensemble prediction algorithm distinguishes the present invention from prior art automated machine learning platforms. Whereas systems such as H2O.ai and DataRobot require human-defined target variables and feature sets, the present invention's ensemble engine operates on autonomously discovered cross-domain pairs with no human specification of which variables to predict or which features to use.

Cross-Layer Enforcement Architecture

Referring to Figure 6, the cross-layer enforcement architecture (Innovation (5)) is a structural constraint embedded at the database query level that ensures all predictions flow from Layer 1 (behavioural signals) to Layer 2 (exploitable outcomes).

Layer Definition:

- **Layer 1 (LAYER1_SOURCES):** Variables with source identifier "wikipedia" or "reddit." These represent fast-moving behavioural signals reflecting human attention and discussion patterns.
- **Layer 2:** All remaining variables (source identifiers: "fred", "stock", "arxiv", "gdelt", and others). These represent slower-moving macroeconomic, financial, academic, and geopolitical outcomes.

Enforcement Mechanism:

The cross-layer constraint is a SQL-level filter applied during the predict_all_pairs query:

```
cross_layer = OR(  
    AND(variable1.source IN LAYER1_SOURCES, variable2.source NOT IN LAYER1_SOURCES),  
    AND(variable2.source IN LAYER1_SOURCES, variable1.source NOT IN LAYER1_SOURCES)  
)
```

This filter is applied as a JOIN condition when selecting pairs for ensemble prediction, ensuring that:

1. Both variables in a pair cannot be Layer 1 (no Wikipedia -> Reddit predictions).
2. Both variables in a pair cannot be Layer 2 (no FRED -> stock predictions).
3. Exactly one variable must be a behavioural signal and one must be an exploitable outcome.

After pair selection, the system assigns roles: the Layer 1 variable becomes the signal (predictor) and the Layer 2 variable becomes the target (predicted). This directional assignment ensures that behavioural data predicts macroeconomic outcomes, not the reverse.

The present invention produces up to two hundred unique cross-layer predictions per scheduled run, filtered from the full set of three thousand seven hundred and twenty-one possible variable pairs.

Exploitation Recommendation Scoring

Referring to Figure 7, the exploitation scoring module (Innovation (2), Innovation (4)) ranks discovered opportunities using a five-component weighted formula. Each exploitation recommendation receives an opportunity score between zero and one hundred, computed as:

$$\text{opportunity_score} = (\text{demand_score} \times 30) + (\text{growth_score} \times 25) + (\text{timing_score} \times 20) + (\text{evidence_score} \times 15) + (\text{competition_score} \times 10)$$

Where the component weights sum to one hundred and each component score is normalised to the range [0, 1]:

- **Demand (weight: 30):** Derived from `estimated_monthly_searches` for the target domain. Higher search volume indicates larger addressable audience.
- **Growth (weight: 25):** Derived from `search_growth_pct`, the percentage change in search volume over the measurement window. Positive growth indicates expanding opportunity.
- **Timing (weight: 20):** Derived from `opportunity_duration_months`, reflecting the estimated window of opportunity. A duration of three to twelve months receives the highest score.
- **Evidence (weight: 15):** Derived from the Granger causality p-value. Lower p-values (stronger causal evidence) yield higher scores.
- **Competition (weight: 10):** Derived from `competition_level` assessment (LOW, MEDIUM, HIGH). Lower competition yields higher scores.

Each recommendation also receives a build viability score that determines whether the recommendation should be autonomously built into an MVP application. Recommendations are classified by action type: BUY, SELL, BUILD, or MONITOR.

The exploitation backtester module validates recommendation quality by measuring actual target variable movement over the opportunity window. For each recommendation with a creation date and target variable, the backtester:

1. Computes the opportunity window: `created_at` to `created_at + (opportunity_duration_months x 30 days)`.
2. Retrieves the target variable value at the window start (baseline) and end (outcome).
3. Calculates actual percentage change: $((\text{outcome} - \text{baseline}) / |\text{baseline}|) \times 100$.
4. Compares predicted direction against actual direction.
5. Backfills `target_growth_actual` and `target_growth_measured_at` on the recommendation record.

The backtester also computes direction accuracy across all measured recommendations and mean absolute error between predicted and actual percentage changes, enabling data-driven evaluation of the scoring formula's effectiveness.

Autonomous TDD Code Generation Pipeline

Referring to Figure 8 and Figure 9, the autonomous code generation pipeline (Innovation (3)) converts exploitation recommendations with sufficient build viability scores into

deployed web applications via a five-phase test-driven development cycle.

Phase 1: Specification Generation (SPEC)

The specification generator analyses the exploitation recommendation text and generates a structured YAML specification comprising:

```
layer_id: LAYER-MVP-{id:04d}
technical_constraints:
  language: Python
  framework: FastAPI
acceptance_criteria:
  - criterion_id: AC-001
    criterion: [description]
    description: [detailed specification]
integration_test_scenarios:
  - scenario: [name]
    description: [specification]
    test_class: [class name]
    tests: [test methods]
e2e_test_scenarios:
  - scenario: [name]
    description: [specification]
    test_class: [class name]
    tests: [test methods]
```

The number of acceptance criteria is determined by the complexity tier:

- **LOW complexity:** four acceptance criteria, one integration test scenario, one end-to-end test scenario.
- **MEDIUM complexity:** seven acceptance criteria, two integration test scenarios, one end-to-end test scenario.
- **HIGH complexity:** twelve acceptance criteria, three integration test scenarios, two end-to-end test scenarios.

Requirement analysis is performed by keyword matching across eight capability categories: frontend, backend, database, authentication, deployment, payments, analytics, and CRUD operations. Template matching scores candidate templates against the requirement using layer alignment (thirty points), tag matching (fifteen points per tag), framework matching (ten points per framework), and description word overlap (five points per keyword, capped at twenty).

Phase 2: Failing Test Creation (RED)

The orchestrator generates comprehensive test files from the YAML specification:

1. Creates test files containing pytest test functions for each acceptance criterion, integration scenario, and end-to-end scenario.
2. Creates a stub module that raises `NotImplementedError` for all imported symbols, enabling pytest collection without implementation.
3. Runs pytest to verify that all tests fail (confirming correct test-implementation separation).

Phase 3: Implementation Generation (GREEN)

The orchestrator generates implementation code that passes the tests:

1. Extracts required symbols from test imports using regex pattern: `^\\s*from\\s+{module}\\s+import\\s+(.+)`.
2. Submits the test code, required symbol list, and (on retries) previous failure output to an AI language model (Anthropic Claude).
3. The AI generates implementation code constrained to define all imported symbols.
4. Runs pytest against the implementation.
5. Computes $pass_rate = tests_passed / (tests_passed + tests_failed + tests_errored)$.
6. Accepts if return code equals zero OR $pass_rate \\geq 0.80$ (eighty percent).
7. On failure, retries up to five times (maximum GREEN phase iterations), carrying forward the failure output as context for the AI.

Phase 4: Code Refinement (REFACTOR)

The orchestrator submits the passing implementation for quality improvement:

1. AI refactors for readability, performance, and maintainability.
2. Pytest is re-run to verify no regressions.
3. If refactored code introduces test failures, the pre-refactor version is retained.

Phase 5: Output Validation (VALIDATE)

The orchestrator performs final validation:

1. Python abstract syntax tree parsing (`ast.parse`) to verify syntactic correctness.
2. Python compilation check (`py_compile`) for all generated files.
3. Final pytest run to confirm all tests pass.
4. File count and line count metrics are recorded.
5. Build status is set to LIVE if all validations pass, or FAILED with error categorisation if any fail.

Error Categorisation: Failures are classified across seven categories: syntax errors, test failures, frontend errors, import errors, wiring errors, configuration errors, and runtime errors. The error breakdown is stored as JSON on the build record to support failure diagnosis in the iterate loop (Innovation (6)).

Algorithm: Five-Phase TDD Code Generation

Input: recommendation (ExploitationRecommendation), complexity_tier (string)
Output: build_record (MVPBuild with status LIVE or FAILED)

Phase 1 - SPEC:

- a. Extract requirement_text from recommendation
- b. Analyse keywords against 8 categories:
KEYWORD_MAPPINGS = {
 'frontend': ['landing', 'page', 'ui', 'website', 'react', 'interface', 'form', 'dashboard'],
 'backend': ['api', 'server', 'endpoint', 'service', 'backend', 'fastapi'],

- ```

'database': ['store', 'save', 'persist', 'database', 'data', 'record', 'user'],
'auth': ['login', 'signup', 'register', 'authenticate', 'auth', 'user', 'password'],
'deployment': ['deploy', 'host', 'railway', 'production', 'serve'],
'payments': ['payment', 'stripe', 'checkout', 'subscription', 'billing', 'saas'],
'analytics': ['analytics', 'tracking', 'ga4', 'mixpanel', 'amplitude', 'metrics'],
'crud': ['create', 'read', 'update', 'delete', 'crud', 'manage', 'list']
}

```
- c. Score candidate templates:
    - score += 30 if layer\_type matches needed category
    - score += 15 per matching tag
    - score += 10 per matching framework
    - score += min(5 x overlapping\_keywords, 20)
  - d. Select highest-scoring template
  - e. Generate YAML spec with acceptance\_criteria count per tier:
    - LOW=4, MEDIUM=7, HIGH=12
    - Plus integration\_test\_scenarios: LOW=1, MEDIUM=2, HIGH=3
    - Plus e2e\_test\_scenarios: LOW=1, MEDIUM=1, HIGH=2

Phase 2 - RED:

- a. Parse YAML spec to extract acceptance criteria and test scenarios
- b. Generate pytest test files with test functions for each criterion
- c. Create stub module: for each imported symbol, define class/function that raises NotImplementedError
- d. Execute: `pytest test_files --tb=short`
- e. Verify: all tests fail (import or NotImplementedError)
- f. If tests unexpectedly pass: flag as spec error

Phase 3 - GREEN (max 5 iterations):

- a. Extract required symbols from test imports:
  - pattern = `r'\s*from\s+{module}\s+import\s+(.+)'`
  - Parse comma-separated symbols from all test files
- b. For attempt = 1 to 5:
  - i. Construct AI prompt containing:
    - Test code (complete, unmodified)
    - Required symbol list (must define all)
    - Complexity tier and acceptance criteria
    - If attempt > 1: previous failure output (stdout + stderr)
  - ii. Submit to AI language model (Anthropic Claude API)
  - iii. Receive generated implementation code
  - iv. Write implementation to temporary file
  - v. Execute: `pytest test_files --tb=short`
  - vi. Parse output: `tests_passed, tests_failed, tests_errored`
  - vii. Compute `pass_rate = tests_passed / (tests_passed + tests_failed + tests_errored)`
  - viii. If `returncode == 0` OR `pass_rate >= 0.80`: accept, break
  - ix. Else: carry forward failure output to next attempt
- c. If all 5 attempts fail: set status = FAILED

Phase 4 - REFACTOR:

- a. Submit passing implementation + test code to AI with refactoring prompt
- b. Receive refactored implementation
- c. Execute: `pytest test_files --tb=short`
- d. If refactored version passes: adopt refactored version
- e. If refactored version fails: retain pre-refactor version (regression guard)

Phase 5 - VALIDATE:

- a. For each generated .py file:
  - i. `ast.parse(source_code)` - verify syntactic correctness
  - ii. `py_compile.compile(file_path)` - verify compilation
- b. Execute final: `pytest test_files -v`
- c. Record metrics: `file_count, total_line_count, test_count`
- d. Classify any remaining errors across 7 categories:
  - `ERROR_CATEGORIES = ['syntax', 'test', 'frontend', 'import', 'wiring', 'config', 'runtime']`
- e. Store `error_breakdown` as JSON on build record
- f. If all validations pass: status = LIVE
- g. Else: status = FAILED with `error_breakdown`

The five-phase TDD pipeline represents a novel approach to autonomous code generation. Unlike AI code generation systems such as GitHub Copilot, which generate code snippets in response to human-authored context, or autonomous agents such as Devin, which require human specification of what to build, the present invention's pipeline receives input from the upstream statistical discovery engine with no human specification. The requirement originates from a statistically validated causal relationship, and the generated application is a direct software embodiment of that statistical signal.

The contract enforcement mechanism - requiring the AI to define all symbols imported by the tests - ensures that generated implementations are structurally compatible with the test suite. The eighty percent pass rate threshold balances completeness against practical viability: a build achieving eighty percent or higher test coverage is likely to be functionally useful even if some edge-case tests fail, while builds below this threshold indicate fundamental implementation failures warranting retry with enriched context.

The import-to-package mapping mechanism automatically resolves common Python package names for generated code:

```
IMPORT_PACKAGE_MAP = {
 'fastapi': 'fastapi', 'uvicorn': 'uvicorn', 'sqlalchemy': 'sqlalchemy',
 'pydantic': 'pydantic', 'requests': 'requests', 'httpx': 'httpx',
 'jinja2': 'Jinja2', 'pytest': 'pytest', 'numpy': 'numpy',
 'pandas': 'pandas', 'stripe': 'stripe', 'anthropic': 'anthropic',
 'yaml': 'PyYAML', 'dotenv': 'python-dotenv', 'jwt': 'PyJWT',
 'passlib': 'passlib'
}
```

This mapping enables the system to automatically generate correct requirements.txt files for deployed applications, further reducing the need for human intervention in the build-to-deploy pipeline.

## Deployment Automation

Referring to Figure 10, the deployment automation module (Innovation (3), Innovation (4)) creates cloud infrastructure and deploys generated applications without human intervention.

### GitHub Repository Creation:

1. Creates a new GitHub repository using PyGithub with auto-initialisation enabled (auto\_init=True).
2. Pushes all generated files using the Git Tree API: creates InputGitTreeElement objects for each file with mode "100644" (regular file) and type "blob."
3. Creates a new commit referencing the tree and updates the default branch reference.
4. Returns the verified commit SHA confirming successful push.

### Railway Cloud Deployment:

1. Creates a new Railway project via GraphQL mutation: projectCreate(input: {name}) -> {id, name}.

2. Creates a service within the project via GraphQL mutation: `serviceCreate(input: {projectId, name}) -> {id, name}`.
3. Deploys the service instance via GraphQL mutation: `serviceInstanceDeploy(serviceId, environmentId, image) -> {id, status}`.
4. Polls deployment status via GraphQL query until status reaches SUCCESS, BUILDING, DEPLOYING, or FAILED.
5. On successful deployment, records the live URL on the build record.

The build status progresses through: QUEUED -> GENERATING -> LIVE (on success) or FAILED (on failure). A stale-build detection mechanism monitors builds that have remained in any active state (QUEUED, GENERATING, UPLOADING, DEPLOYING) for longer than six hundred seconds and marks them as FAILED to prevent indefinite resource consumption.

### **Outcome Measurement and Feedback Loop**

Referring to Figure 11, the outcome measurement subsystem (Innovation (2), Innovation (4)) closes the loop between prediction and reality.

#### **Prediction Validation (Daily at 02:30 UTC):**

1. Queries all PredictionTracking records with status = 'pending' and target\_date <= current date.
2. For each pending prediction, fetches the actual value of the target variable at or near the target date from the TimeSeriesData table.
3. Computes actual direction (up or down), actual percentage change, and value error percentage.
4. Sets direction\_correct = True if predicted\_direction equals actual\_direction.
5. Updates status to 'validated.'
6. Stores R2 (coefficient of determination) where applicable.

#### **Walk-Forward Backtesting (Weekly on Monday at 04:00 UTC):**

1. For each signal-target pair with sufficient data, performs expanding-window OLS backtesting.
2. Starting from split index six (minimum training window), iterates through all subsequent data points.
3. At each split: trains OLS on data[0:split\_idx], predicts the next value, records direction correctness.
4. Stores each backtest prediction as a PredictionTracking record with model\_version = 'backtest\_walkforward.'
5. Computes aggregate accuracy metrics per pair.

#### **Ensemble Weight Recalibration:**

Upon accumulation of ten or more validated predictions for a signal-target pair, the `calibrate_weights` function computes per-model accuracy from `PredictionTracking` records grouped by `model_version` ('granger\_v1', 'ols\_v1', 'arima\_v1'). Updated weights are applied to subsequent predictions, enabling the ensemble to automatically adapt to which sub-model performs best for each specific pair.

### Exploitation Backtesting:

The exploitation backtester measures actual outcomes for all exploitation recommendations by comparing predicted direction and score against actual target variable movement over the opportunity window. Component-level accuracy analysis identifies which scoring components (demand, growth, timing, evidence, competition) best discriminate between successful and unsuccessful recommendations, generating weight adjustment recommendations.

### Build-Iterate Loop with Failure Diagnosis

Referring to Figure 12, the build-iterate loop (Innovation (6)) enables self-correcting code generation.

When a build fails any TDD phase, the system:

1. **Diagnoses the failure:** Classifies errors across seven categories (syntax, test, frontend, import, wiring, configuration, runtime) and stores the breakdown as JSON.
2. **Generates iteration context:** Collects the parent build's error patterns, generated file contents, and latest ensemble predictions for related signals.
3. **Spawns a child build:** Creates a new `MVPBuild` record with:
  - `parent_build_id` = source build identifier
  - `iteration_number` = parent iteration number + 1
  - `iterate_reason` = diagnostic summary of what to fix
4. **Enriches the specification:** The child build's AI prompt includes the parent's error breakdown and failing test output, enabling the AI to generate corrective code.
5. **Executes the pipeline:** The child build proceeds through the full five-phase TDD cycle with the enriched context.

A concurrency guard prevents multiple simultaneous iterations in the same chain: if an active build (status `QUEUED` or `GENERATING`) already exists in the iteration chain, the iterate request is blocked.

The iteration chain is queryable, returning all builds sharing a common root ancestor ordered by iteration number. This enables tracking of the system's self-correction trajectory - how many iterations were required to achieve a `LIVE` deployment, and which error categories were resolved at each stage.

### Algorithm: Build Iteration with Failure Diagnosis

Input: `parent_build` (`MVPBuild` with status `FAILED` or `LIVE`)

Output: child\_build (MVPBuild) or error

Step 1: Validation

- a. Verify parent\_build exists and status in {LIVE, FAILED}
- b. Query iteration chain: all MVPBuild records reachable via parent\_build\_id
- c. If any build in chain has status in {QUEUED, GENERATING}: return error  
"Active build already exists in iteration chain" (concurrency guard)

Step 2: Failure Diagnosis

- a. Parse parent\_build.error\_breakdown (JSON):

```
{
 "syntax": count_syntax_errors,
 "test": count_test_failures,
 "frontend": count_frontend_errors,
 "import": count_import_errors,
 "wiring": count_wiring_errors,
 "config": count_config_errors,
 "runtime": count_runtime_errors
}
```
- b. Identify predominant\_category = category with highest count
- c. Extract specific error messages from build step output

Step 3: Context Assembly

- a. Collect parent error\_breakdown (complete JSON)
- b. Collect parent generated files (source code + tests)
- c. Query latest ensemble predictions for signal-target pair:  
PredictionTracking where signal = parent.signal\_name  
AND target = parent.target\_name, ordered by creation date
- d. Assemble iterate\_reason = "Fix: {predominant\_category} errors  
{count}}, plus {secondary\_category} ({count})"

Step 4: Child Build Creation

- a. Create new MVPBuild record:  
parent\_build\_id = parent.id  
iteration\_number = parent.iteration\_number + 1  
iterate\_reason = assembled reason string  
status = QUEUED  
recommendation\_id = parent.recommendation\_id
- b. Enrich the build specification prompt with:
  - Parent's complete error\_breakdown
  - Parent's failing test output (stdout + stderr)
  - Updated ensemble prediction context
  - Instruction: "This is iteration {N}. The previous build failed with {predominant\_category} errors. Focus on resolving these specific issues."

Step 5: Pipeline Execution

- a. Submit child build to the five-phase TDD pipeline (Figure 8)  
as a background task
- b. Return child build record with status QUEUED

The build-iterate loop represents Innovation (6) and is the mechanism by which the system achieves self-healing code generation. Each iteration carries forward the diagnostic context from its predecessor, enabling the AI language model to generate progressively more correct implementations. The concurrency guard prevents resource waste from parallel iterations that might conflict, ensuring that only one active generation attempt exists in each iteration chain at any time.

The iteration chain data structure enables both retrospective analysis (which error categories are most common across all builds) and per-chain analysis (how many iterations are typically required to resolve specific error types). This data feeds into the broader outcome measurement subsystem, enabling the system to learn not only which statistical signals are predictive but also which types of software applications are most

amenable to autonomous generation.

### Autonomous Scheduling Timeline

Referring to Figure 13, the system's autonomous loop executes on the following daily schedule (all times UTC):

Time Frequency Operation Purpose

01:00 Daily Wikipedia data ingestion Fetch daily and monthly pageview data (Layer 1)

01:30 Daily Reddit data ingestion Fetch thirty-day activity window (Layer 1)

02:00 Daily GDELT data ingestion Fetch thirty-day event tone data (Layer 2)

02:30 Daily Update prediction actuals Validate matured predictions against real values

03:00 Weekly (Sunday) Snapshot baselines Record model accuracy metrics for trend analysis

04:00 Weekly (Monday) Walk-forward backtest Expanding-window OLS backtesting for all pairs

05:00 Daily Ensemble predictions Run all cross-layer predictions (maximum two hundred pairs)

This schedule ensures that data freshness propagates through the pipeline before predictions are generated. The thirty-minute spacing between ingestion tasks prevents API rate limit contention. Prediction validation at 02:30 runs before new predictions at 05:00, ensuring that the weight calibration mechanism has the most recent accuracy data available.

### Data Model Architecture

The system's relational database schema comprises the following principal entities that support the autonomous pipeline:

**VariableMetadata:** Stores the canonical definition of each tracked variable, comprising: unique identifier, variable name, data source identifier (e.g., "wikipedia", "reddit", "fred", "stock", "arxiv", "gdelt"), human-readable description, units of measurement, data collection frequency, and active status flag. The source identifier field is the basis for cross-layer enforcement (Innovation (5)): the system partitions variables into Layer 1 (source in LAYER1\_SOURCES) and Layer 2 (all other sources) based solely on this field.

**TimeSeriesData:** Stores all collected time-series observations, comprising: unique identifier, foreign key to VariableMetadata, timestamp, and numerical value. Indexed by (variable\_id, timestamp) for efficient time-range queries during correlation computation and prediction validation.

**CorrelationResult:** Stores the unified statistical profile for each variable pair (Innovation (1)), comprising: foreign keys to both VariableMetadata records, Pearson correlation value, Spearman correlation value, Kendall correlation value, p-value (minimum across the three coefficients), absolute correlation value (for ranking), sample size (number of aligned

data points), significance flag ( $p < 0.05$ ), source identifiers for both variables, Granger p-value for the forward test ( $X \rightarrow Y$ ), Granger p-value for the reverse test ( $Y \rightarrow X$ ), optimal Granger lag count, and causal direction category ( $x\_to\_y$ ,  $y\_to\_x$ , bidirectional, none). This table serves as the input for ensemble pair selection.

**PredictionTracking:** Stores all predictions and their validation outcomes (Innovation (2), Innovation (4)), comprising: unique prediction identifier (UUID), signal variable name, target variable name, predicted direction (up or down), predicted value, predicted change percentage, actual direction (populated upon validation), actual value, actual change percentage, direction correctness flag, R-squared coefficient of determination, confidence score, model version identifier (e.g., "granger\_v1", "ols\_v1", "arima\_v1", "ensemble\_v1", "backtest\_walkforward"), status (pending, validated, expired), target date, and creation timestamp. Indexed by (status, target\_date) for efficient validation queries and by (signal\_name, target\_name, model\_version) for weight calibration queries.

**ExploitationRecommendation:** Stores scored exploitation opportunities, comprising: action type (BUY, SELL, BUILD, MONITOR), opportunity score (0-100), build viability score (0-100), estimated monthly searches, search growth percentage, opportunity duration in months, revenue potential level (LOW, MEDIUM, HIGH), competition level (LOW, MEDIUM, HIGH), predicted direction, predicted change percentage, target variable name, target growth actual (backfilled upon measurement), and target growth measured-at timestamp.

**MVPBuild:** Stores the complete state of each autonomous build (Innovation (3), Innovation (6)), comprising: unique identifier, foreign key to ExploitationRecommendation, status (QUEUED, GENERATING, UPLOADING, DEPLOYING, LIVE, FAILED), complexity tier, duration in seconds, AI cost in USD, file count, total line count, total error count, error breakdown (JSON with counts per seven error categories), build steps (JSON timeline), generated files (JSON or stored references), GitHub repository URL, Railway deployment URL, iteration number (default 1), parent build identifier (nullable, for iteration chains), iterate reason (nullable), and timestamps for creation and completion.

### Example Scenario: Wikipedia Signal -> Stock Prediction -> MVP Build

Referring to Figure 14, the following example illustrates the complete autonomous pipeline:

**Step 1: Data Ingestion.** At 01:00 UTC, the Wikipedia ingestion task fetches daily pageview data for "Artificial\_intelligence" and stores it as a Layer 1 variable. At 02:00 UTC, the system has also ingested FRED data including the NASDAQ composite index as a Layer 2 variable.

**Step 2: Correlation Discovery.** The correlation engine computes Pearson correlation between Wikipedia "Artificial\_intelligence" pageviews and NASDAQ composite index values, finding  $r = 0.72$  with  $p < 0.001$  over sixty aligned data points.

**Step 3: Granger Causality Testing.** The Granger module tests bidirectionally with monthly frequency (maximum twelve lags). The forward test (Wikipedia -> NASDAQ) yields  $p = 0.03$  at lag three, while the reverse test (NASDAQ -> Wikipedia) yields  $p = 0.28$ . Causal direction is assigned as  $x\_to\_y$ : Wikipedia pageviews Granger-cause NASDAQ

movement with a three-month lead.

**Step 4: Ensemble Prediction.** The ensemble engine generates a prediction:

- Granger sub-model: direction = "up", confidence = 0.97 (from 1 - 0.03).
- OLS sub-model: positive slope, direction = "up", confidence = 0.68 (from  $R^2 = 0.68$ ).
- ARIMA sub-model: forecast above current value, direction = "up", confidence = 0.55.
- Ensemble result:  $\text{up\_score} = (0.40 \times 0.97) + (0.35 \times 0.68) + (0.25 \times 0.55) = 0.765$ . Direction = "up", confidence = 0.765.

**Step 5: Exploitation Scoring.** The exploitation module generates a recommendation:  $\text{action\_type} = \text{"BUILD"}$ ,  $\text{opportunity\_score} = 78$  (demand:  $0.85 \times 30$  + growth:  $0.60 \times 25$  + timing:  $0.70 \times 20$  + evidence:  $0.97 \times 15$  + competition:  $0.30 \times 10 = 25.5 + 15.0 + 14.0 + 14.55 + 3.0 \approx 72$ ).

**Step 6: Autonomous Build.** The build pipeline receives the recommendation: "Build a web application that displays AI-sector stock predictions based on Wikipedia interest trends." The specification generator produces a YAML spec with seven acceptance criteria (MEDIUM complexity). The TDD pipeline generates tests, creates a FastAPI application with prediction display and historical charts, achieves ninety-two percent test pass rate, and validates the output.

**Step 7: Deployment.** The deployment module creates a GitHub repository "ai-stock-predictor-mvp," pushes the generated code, creates a Railway project, and deploys the application. The build record is updated with status LIVE and the public URL.

**Step 8: Outcome Measurement.** Three months later (matching the Granger lag), the prediction validation task compares the predicted NASDAQ direction ("up") against the actual movement. If correct, the  $\text{direction\_correct}$  field is set to True and the Granger sub-model's track record improves, increasing its calibrated weight for this pair in future predictions.

## Alternative Embodiments

Those skilled in the art will appreciate that the present invention may be implemented in various alternative forms without departing from the scope of the claims.

**Alternative Data Sources.** While the preferred embodiment ingests data from Wikipedia, Reddit, FRED, stock markets, ArXiv, and GDELT, the system architecture supports any combination of heterogeneous time-series data sources. Alternative behavioural signal sources include Google Trends search volume data, Twitter/X social media engagement metrics, YouTube view counts, App Store download statistics, and web traffic analytics. Alternative outcome sources include commodity prices, cryptocurrency exchange data, real estate indices, labour market statistics, patent filing volumes, and clinical trial registries. The cross-layer enforcement mechanism operates on source category labels and is agnostic to the specific data sources employed.

**Alternative Ensemble Configurations.** While the preferred embodiment employs three sub-models (Granger, OLS, ARIMA), the ensemble architecture supports any number of statistical sub-models with weighted voting. Alternative sub-models include vector

autoregression (VAR), exponential smoothing (ETS), Prophet time-series decomposition, gradient-boosted tree regressors, long short-term memory (LSTM) neural networks, and Bayesian structural time-series models. The weight calibration mechanism operates identically regardless of the number or type of sub-models, requiring only that each sub-model produce a direction and confidence score.

**Alternative Deployment Targets.** While the preferred embodiment deploys to GitHub for code hosting and Railway for cloud deployment, the system supports alternative platforms including Amazon Web Services (AWS Lambda, Elastic Beanstalk), Google Cloud Platform (Cloud Run, App Engine), Microsoft Azure (Azure Functions, App Service), Heroku, Vercel, and Netlify. The deployment automation module requires only API access for repository creation and application deployment, and may be extended to support containerised deployment via Docker and Kubernetes.

**Alternative Test Frameworks and Languages.** While the preferred embodiment generates Python code tested with pytest, the test-driven development pipeline supports alternative programming languages and test frameworks including JavaScript with Jest or Mocha, TypeScript with Vitest, Ruby with RSpec, and Go with the standard testing package. The pipeline's contract enforcement mechanism (requiring all imported symbols to be defined) is language-agnostic in principle.

**Alternative AI Language Models.** While the preferred embodiment uses Anthropic Claude for code generation, the pipeline supports alternative large language models including OpenAI GPT models, Google Gemini, Meta Llama, and Mistral. The AI interaction interface requires only a text-in, text-out API and is not coupled to any specific model provider.

## **INDUSTRIAL APPLICABILITY**

The present invention has industrial applicability in financial technology, market research, product development automation, and autonomous business operations. The system enables organisations to systematically discover actionable cross-domain relationships and automatically exploit them through generated software products - reducing the time from insight discovery to product deployment from weeks or months to hours. The autonomous feedback loop enables continuous improvement without manual intervention, making the system suitable for deployment in environments requiring twenty-four-hour-a-day, seven-day-a-week autonomous operation such as financial markets, real-time demand forecasting, and autonomous portfolio management.